Tomasulo Algorithm: Multi-Issue Simulator

Ahmed Ahmed
Computer Architecture | Revised October 25, 2026

1 Introduction

I extended the supplied Python Tomasulo simulator to issue one, two, three, or four instructions per cycle. My main goal was to increase issue bandwidth without changing the program result or losing the ordering rules needed for speculation. I kept issue in program order, allowed ready instructions to execute out of order, and retired instructions in order.

I used the sample report as a guide for showing code changes, test inputs, and cycle tables. In this version I include the complete committed instruction traces for all seven benchmarks at all four issue widths. Each case also has its hardware settings, initial state, final state, and my explanation of the result.

What I found in the review

The review confirmed the main multi-issue, renaming, branch-recovery, and memory-ordering behavior in the tested cases. It also exposed a precision gap: I had stored memory values as ordinary Python floats. I corrected the memory path to use single-precision values for initialization, stores, loads, and forwarding, while keeping the .d arithmetic in the wider floating-point registers.

After the correction, I reran 55 automated test methods, including 120 seeded differential configurations, and all 28 benchmark configurations. All passed. I checked final state with a separate sequential interpreter and checked cycle timing with dedicated assertions.

How I organized the report

Section	Evidence
2 Requirements review	A checklist linking each requirement to code and tests; assumptions are stated explicitly.
3 Code changes	Actual starter and implemented source excerpts, including the precision fix found in this review.
4 Test results	All seven inputs, configurations, final states, and all 28 committed timing tables.
5 Verification and limitations	Test coverage, focused recovery evidence, reproduction commands, and remaining modeling assumptions.

I do not use a passing test suite as a proof for every possible input. The checklist distinguishes implemented behavior from details that the assignment leaves open.

2 Requirements review - processor and input

I checked the implementation against the supplied assignment, rather than treating every behavior in the sample report as a requirement. The sample uses a wider writeback/commit policy; my simulator retains the assignment's one CDB. A method name below refers to tomasulo/engine.py unless another file is named.

Requirement	Implementation and evidence	Finding
In-order issue widths 1-4; default 1	Config.issue_width and _issue process a bundle sequentially. Input and CLI select the width. Tests check the width bound, default, overrides, full RS, and full ROB.	Implemented / tested
ISSUE, EX, MEM, WB, COMMIT	run separates stage selection/completion. Each dynamic trace records ISSUE, EX/MEM intervals, WB, and COMMIT. Simple-add and multicycle tests check boundaries.	Implemented / tested
32 integer + 32 FP registers; R0=0	Simulator initializes both files; source capture and commit protect R0. Tests cover initialization, writes to R0, and separate R/F names.	Implemented / tested
RAT, reservation stations, ROB, one CDB	_source, _issue, _writeback, and _commit use producer tags and ordered retirement. Bundle RAW/WAR/WAW and recovery tests check ownership. _writeback selects one result.	Implemented / tested
Instruction buffer / fetch and decode	Program.instructions holds decoded instructions and pc selects the next fetch inside ISSUE. There is no separate cycle or bounded prefetch queue; the handout specifies no buffer size.	Equivalent representation
Integer unpipelined; FP units pipelined	_start_execution limits active integer/address units, but admits one FP operation per configured FU per cycle. Dedicated tests cover both FP units and FU counts.	Implemented / tested
Configurable ROB, RS, FU counts, latencies	Config.validate and parser.py accept the configuration table and reject nonpositive settings. Resource-pressure, small-ROB/RS, latency, and multiple-FU tests exercise them.	Implemented / tested
All ten ISA operations; case-insensitive input	parser._instruction accepts Ld, Sd, Beq, Bne, Add, Sub, Addi, Add.d, Sub.d, Mult.d. ISA, parser, initialization, and malformed-input tests cover them.	Implemented / tested
Default zero state; register/memory input	Parser reads optional initializers; Simulator supplies omitted zeros. Tests check full final state and edge addresses.	Implemented / tested

2 Requirements review - speculation, memory, delivery

Requirement	Implementation and evidence	Finding
8-entry BTB; 1-bit direction; index low 3 word-PC bits	The BTB has eight tagged slots, indexed by pc % 8. Prediction happens in _issue; branch outcomes train it at EX completion. Loop and BTB-collision tests cover reuse.	Implemented / tested
Branch in integer RS; target PC+4+(offset<<2)	kind routes branches to int_unit. Word target pc+1+offset is equivalent to the required byte-PC rule. Beq/Bne and loop tests exercise the result.	Implemented / tested
Recovery at EX end; correct fetch n+2	_squash removes younger ROB, RS/operand, execution, LSQ, and memory work; _rebuild_rat restores survivors. fetch_resume = cycle+2. Tests check timing and wrong-path effects.	Implemented / tested
Ordered LSQ; separate address adder	_issue appends memory tags in order. lsu executes effective addresses independently of the integer ALU. Load-queue tests cover release and capacity.	Implemented / tested
Store at commit; dequeue store only then	_start_memory starts a store only at the ROB head; _commit performs the write and frees the LSQ entry. Wrong-path store tests check precise state.	Implemented / tested
Load memory access / one-cycle forwarding	_load_value selects the youngest matching older store and waits for its data. A forwarded request uses one cycle. Completed loads leave the LSQ before retirement.	Implemented / tested
Unresolved stores may be bypassed	Loads may read before older addresses resolve. _check_memory_violation replays stale reads; an intervening matching store prevents unnecessary replay.	Implemented / tested
Single-port, nonpipelined 256B / 64W memory	One memory_job serves loads, stores, or forwarding. Addresses are byte values 0,4,...,252. Port serialization is tested; alignment is an explicit interpretation.	Implemented; alignment assumption
Single-precision Ld/Sd data	Memory values and forwarded store data are rounded to binary32. .d register arithmetic remains Python binary64. Independent constant-based precision tests cover all widths.	Gap fixed / tested
Required output, tests, README, GitHub report	output.py prints per-stage cycles, both register files, and nonzero memory; JSON preserves squashed attempts. main.py, benchmarks/, tests/, README.md, results/, and REPORT.pdf form the submission.	Included

3 Code changes - overview and issue width

Brief description what you have added to the code

I kept the original modules in starter_original/ and refactored the running simulator into a small package. model.py defines the input model; parser.py reads it; engine.py manages dynamic instructions; output.py formats results; and main.py provides the command line. I also added a separate interpreter in tests/oracle.py and the repeatable benchmark runner.

The following are literal excerpts from the submitted files. I removed only shared leading indentation for readability. These are selected sections of functions, so the source labels identify where the complete code lives. The old and new filenames differ because of the refactor.

3.1 Configurable issue width and ordered dispatch

I replaced the single issue call with a loop limited by the configured issue width. I still check reservation-station and ROB capacity before each instruction. If the next instruction cannot issue, I stop the loop. This lets widths 1-4 dispatch several instructions per cycle while preserving program order.

Initial version

starter_original/main.py | lines 95-102

```
'''ISSUE stage'''
if (PC.PC<len(instructions))&(PC.valid==1):
    issue(cycle, PC, instructions, ROB, size_ROB,
            rs_int_adder,
            rs_fp_adder,
            rs_fp_multi,
            ld_sd_queue, size_ld_sd_queue,
            rat_int, rat_fp)
```

My version

tomasulo/model.py | lines 20-27

```
class Config:
    issue_width: int = 1
    rob_entries: int = 128
    int_unit: UnitConfig = field(default_factory=lambda: UnitConfig(2, 1))
    fp_add: UnitConfig = field(default_factory=lambda: UnitConfig(3, 3))
    fp_mul: UnitConfig = field(default_factory=lambda: UnitConfig(2, 20))
    lsu: UnitConfig = field(default_factory=lambda: UnitConfig(3, 1))
    mem_cycles: int = 4
```

tomasulo/engine.py | lines 115-127

```
for _ in range(self.config.issue_width):
    if self.pc == len(self.program.instructions):
        break
    if not 0 <= self.pc < len(self.program.instructions):
        raise ValueError(f'Instruction PC {self.pc} is outside the program')
    instruction = self.program.instructions[self.pc]
    kind = self.kind(instruction.op)
    unit = getattr(self.config, kind)
    occupied = (len(self.lsq) if kind == 'lsu' else
                sum(e.kind == kind and e.state == 'waiting' for e in self.rob))
    if len(self.rob) >= self.config.rob_entries or occupied >= unit.rs:
        self.stats['issue_stall_cycles'] += 1
        break  # Ordered issue: never skip a blocked instruction.
```

3 Code changes - renaming and cycle timing

3.2 Source operands and safe RAT updates

I kept source capture before destination renaming, which the starter already did. I changed the unsafe broadcast behavior: an older writer no longer replaces a newer RAT mapping. My RAT holds producer tags until retirement, and I remove a mapping only when the committing instruction still owns it.

Initial version

starter_original/wb.py | lines 25-31

```
reg_tag = ROB[index].dest_tag
# rat_int
if reg_tag[0] == 'R':
    rat_int[int(reg_tag[1:])] = cdb.value
# rat_fp
if reg_tag[0] == 'F':
    rat_fp[int(reg_tag[1:])] = cdb.value
```

My version

tomasulo/engine.py | lines 128-128

```
operands = [self._source(r) for r in instruction.sources]
```

tomasulo/engine.py | lines 143-144

```
if instruction.dest and instruction.dest != 'R0':
    self.rat[instruction.dest] = entry.tag
```

tomasulo/engine.py | lines 377-381

```
dest = entry.instruction.dest
if dest and dest != 'R0':
    self.registers[dest] = entry.value
    if self.rat.get(dest) == entry.tag:
        del self.rat[dest]
```

3.3 One execution advance per cycle and explicit readiness

I replaced incremented execution counters with an absolute completion cycle. The starter could call the same functional-unit update before and after adding a waiting instruction, advancing active entries twice. I also check both issue time and operand readiness, so an instruction cannot execute during its issue cycle or consume a same-cycle broadcast.

Initial version

starter_original/exe.py | lines 120-121

```
# functional units
fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
```

starter_original/exe.py | lines 144-146

```
# check if it's a waiting instruction
if ROB[find_ROB_entry(ROB, rs_int_adder[index].dest_tag)].issue[0] < cycle:
    fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
```

My version

tomasulo/engine.py | lines 156-158

```
def _operands_ready(self, entry, address_only=False):
    operands = entry.operands[-1:] if address_only else entry.operands
    return all(o.tag is None and o.ready_cycle < self.cycle for o in operands)
```

tomasulo/engine.py | lines 171-179

```
if (entry.kind != kind or entry.state != 'waiting' or
        entry.trace['issue'] >= self.cycle or
        not self._operands_ready(entry, kind == 'lsu')):
    continue
entry.state = 'executing'
entry.trace['ex_start'] = self.cycle
end = self.cycle + unit.ex_cycles - 1
entry.trace['ex_end'] = end
self.executing.append((end, entry.tag))
```

3 Code changes - branch prediction and recovery

3.4 Branch prediction and recovery after a wrong prediction

I changed branches from stopping fetch to using an eight-entry, direct-mapped branch target buffer. A missing entry follows the next instruction. When a prediction is wrong, I squash younger work, cancel its execution or memory request, rebuild the RAT from surviving ROB entries, and restart fetch at the resolved PC.

Initial version

starter_original/issue.py | lines 239-244

```
put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
# put ins into rs_int_adder
index = check_rs_space(rs_int_adder)
put_bne_into_rs(rs_int_adder, index, ins, rat_int, rat_fp, ROB)
# PC
PC.valid = 0
```

My version

tomasulo/engine.py | lines 146-153

```
if instruction.op in ('beq', 'bne'):
    prediction = self.btb[self.pc % 8]
    if prediction and prediction['pc'] == self.pc and prediction['taken']:
        next_pc = prediction['target']
    entry.predicted_pc = next_pc
    trace['predicted_pc'] = next_pc
    self.stats['branch_predictions'] += 1
self.pc = next_pc
```

tomasulo/engine.py | lines 196-211

```
def _squash(self, first_tag, new_pc, reason):
    removed = [e for e in self.rob if e.tag >= first_tag]
    if removed:
        for entry in removed:
            entry.trace.update(squashed=True, squash_cycle=self.cycle,
                               squash_reason=reason)
            self.entries.pop(entry.tag)
        self.stats['squashed'] += len(removed)
        self.rob = [e for e in self.rob if e.tag < first_tag]
        self.lsq = [t for t in self.lsq if t < first_tag]
        self.executing = [(end, t) for end, t in self.executing if t < first_tag]
        if self.memory_job and self.memory_job[1] >= first_tag:
            self.memory_job = None
    self._rebuild_rat()
    self.pc = new_pc
    self.fetch_resume = self.cycle + 2
```

The predictor follows the fall-through path on an empty slot. I store a word-PC tag as well as the target so two branches with the same three index bits do not use each other's prediction. A wrong prediction resolves at EX end, recovery occupies the next cycle, and fetch can resume one cycle after that.

3 Code changes - forwarding and memory replay

3.5 Forward from the youngest older store with a matching address

I corrected the store search used by loads. The starter stops at the nearest older store even if that store has a different address. I scan backward until I find the youngest older store with the same address. If its data is not ready, the load waits instead of using stale memory.

Initial version

starter_original/mem.py | lines 19-27

```
def forward_check_from_sd(ld_sd_queue, index):
    data = []
    for i in reversed(range(index)):
        if (ld_sd_queue[i].op=='Sd'):
            if (ld_sd_queue[i].address == ld_sd_queue[index].address):
                if (type(ld_sd_queue[i].data)==int)|(type(ld_sd_queue[i].data)==float):
                    data.append(ld_sd_queue[i].data)
            break
    return data
```

My version

tomasulo/engine.py | lines 286-293

```
def _load_value(self, load):
    for tag in reversed(self.lsq):
        older = self.entries[tag]
        if tag < load.tag and older.instruction.op == 'sd' and older.address == load.address:
            if not self._operands_ready(older):
                return False, None, False
            return True, older.operands[0].value, True
    return True, None, False
```

3.6 Replaying a load after an address conflict

I allow loads to pass stores whose addresses are still unknown, as the assignment requests. If an older store later resolves to an address already read by a younger load, I discard that load and its successors and fetch again. A known matching store in between can supersede the older store, so I check for that case before replaying.

Initial version

starter_original/mem.py | lines 30-40

```
def check_all_previous_Sd_no_match(ld_sd_queue, index):
    flag = True
    for i in range(index):
        # all Sd instructions
        if (ld_sd_queue[i].op=='Sd'):
            if ld_sd_queue[i].ready == 0:
                flag = False
                break
            elif (ld_sd_queue[i].address==ld_sd_queue[index].address):
                flag = False
                break
```

My version

tomasulo/engine.py | lines 213-229

```
def _check_memory_violation(self, store):
    # Loads leave the LSQ on receiving data but remain in the ROB until
    # retirement, so all speculative reads can still be checked here.
    for younger in self.rob:
        if (younger.tag > store.tag and younger.instruction.op == 'ld' and
                younger.trace['mem_start'] is not None and
                younger.address == store.address):
            # A known intervening same-address store supersedes this store.
            intervening = any(e.tag > store.tag and e.tag < younger.tag and
                              e.instruction.op == 'sd' and
                              e.address == store.address for e in self.rob)
            if intervening:
                continue
            younger.trace['replays'] += 1
            self.stats['memory_replays'] += 1
            self._squash(younger.tag, younger.instruction.pc, 'memory replay')
            return
```

3 Code changes - input, output, and validation

3.7 New command-line options and configuration validation

I added command-line options for the input file, issue width, cycle limit, and saved results. I also validate configuration values before simulation starts, including positive integers and the supported width range. These are new interfaces; the starter used hardcoded settings and only read assembly lines from code.in.

Initial version

The starter has no equivalent argument parser or centralized configuration validator. It reads code.in from a fixed path and defines hardware constants in main.py.

My version

main.py | lines 14-20

```
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument('input', nargs='?', default='code.in', help='assembly/configuration input')
parser.add_argument('--issue-width', type=int, choices=range(1, 5), help='override input width (1-4)')
parser.add_argument('--max-cycles', type=int, default=100000)
parser.add_argument('--json', dest='json_path', type=Path, help='write complete machine-readable results')
parser.add_argument('--output', type=Path, help='also save the text results')
parser.add_argument('--show-squashed', action='store_true', help='include wrong-path and replayed rows')
```

tomasulo/model.py | lines 32-40

```
def positive_integer(name: str, value: int) -> None:
    if type(value) is not int or value <= 0:
        raise ValueError(f"{name} must be a positive integer")

positive_integer("issue width", self.issue_width)
if self.issue_width > 4:
    raise ValueError("issue width must be between 1 and 4")
positive_integer("ROB entries", self.rob_entries)
positive_integer("memory cycles", self.mem_cycles)
```

I added readable text output for the status table, registers, and nonzero memory. JSON preserves every dynamic attempt, including squashed instructions, so I can inspect branches and replay without confusing those attempts with committed instructions.

I test functional correctness against a sequential interpreter that has its own input parsing and architectural execution. It does not share Tomasulo scheduling or renaming code. I also use direct timing assertions, because a correct final result alone cannot tell me whether the simulated stage cycles are correct.

3.8 Single-precision memory correction

During this review I found that memory transfers kept Python binary64 values unchanged. That did not fully implement the assignment's single-precision Ld/Sd wording. I now round initialized memory and store data to binary32, and use the same rounded value on the forwarding path. The source FP register stays unchanged.

For example, storing F2=16777217 must produce 16777216 in memory and in a forwarded or later loaded value. Keeping the wider value only on the forwarding path would make results depend on scheduling. My new tests force both paths at every issue width.

3 Code changes - precision correction in source

Before this review

The following three statements are the exact replaced statements from the previously submitted engine. The original file is preserved in the preceding GitHub commit f8caa84d.

```
self.memory.update(program.memory)
entry.value = self.memory[entry.address] if value is None else value
self.memory[entry.address] = self.memory_job[2]
```

My corrected version

tomasulo/numeric.py | lines 1-17

```
"""Numeric representation for the assignment's 32-bit data-memory words.

Floating-point registers and .d arithmetic use Python's binary64 floats. Ld
and Sd transfer single-precision values, so all memory paths round to binary32
using round-to-nearest, ties-to-even. A rounded overflow becomes signed infinity.
"""

import math
import struct


def memory_word(value: int | float) -> float:
    """Return a binary32 value, represented exactly in a Python float."""
    try:
        return struct.unpack("!f", struct.pack("!f", value))[0]
    except OverflowError:
        return math.copysign(math.inf, value)
```

tomasulo/engine.py | lines 72-73

```
self.memory.update({address: memory_word(value)
                    for address, value in program.memory.items()})
```

tomasulo/engine.py | lines 331-331

```
entry.value = memory_word(self.memory[entry.address] if value is None else value)
```

tomasulo/engine.py | lines 367-367

```
self.memory[entry.address] = memory_word(self.memory_job[2])
```

I leave Add.d, Sub.d, and Mult.d arithmetic in the FP registers at Python float precision. The model does not implement floating-point control registers, exception flags, or alternative rounding modes. These are separate from the single-precision memory transfer that I corrected.

4 Results across issue widths

Show results of test cases under different issue width

I ran these experiments on October 25, 2026. I held each benchmark's program, initial state, and hardware settings fixed while changing only the issue width. I compared every final register and every memory word with the independent sequential interpreter, not just the values shown in the report.

Benchmark	W1	W2	W3	W4	W4 speedup
Corrected starter loop	66	65	65	65	1.015x
Independent mixed operations	47	48	47	47	1.000x
Dependency / renaming chain	40	38	38	36	1.111x
Store-to-load forwarding	44	44	44	44	1.000x
Branch recovery	44	44	44	44	1.000x
Memory replay	34	34	35	35	0.971x
Small ROB / RS / LSQ	42	42	42	42	1.000x

Table: total cycles. Every one of the 28 configurations passed the final-state check. Speedup is width-1 cycles divided by width-4 cycles.

The dependency case improved from 40 to 36 cycles, a 10% reduction. The corrected loop improved from 66 to 65. The forwarding, branch, and resource-pressure cases did not improve. The replay case increased from 34 to 35 cycles at widths 3 and 4. I therefore cannot conclude that a larger issue width always makes a program faster.

A wider issue stage admits more instructions but still competes for one CDB, one commit slot, and one memory service. Data dependencies, recovery, queue capacity, and the order in which ready work reaches these shared resources all affect the result. The next pages show the cycle-level evidence for each case.

How to read the timing tables

I show dynamic committed instructions in program order. Repeated PCs are different loop iterations. Dynamic IDs can skip because younger speculative attempts were squashed. EX and MEM ranges are inclusive. A dash means the instruction does not use that stage. A store's MEM range is the memory service performed at the ROB head, and COMMIT is the cycle when it writes memory. Branches resolve in EX and do not use the CDB.

The JSON files include all squashed attempts as well. A squashed instruction can have a planned EX/MEM end cycle that it never reaches. I keep those rows out of the committed tables and use separate recovery evidence later in this report.

4.1 Corrected starter loop

I used the supplied loop to combine loads, FP multiplication, stores, integer updates, and a backward branch. I corrected its branch offset from -12 to -8 under the required word-offset rule; the original input is preserved in benchmarks/.

Input: benchmarks/corrected_starter.in. I kept the hardware fixed for widths 1-4.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	3	4	-	1
FP multiplier	2	15	-	1
Load/store address unit	5	1	5	1

ROB entries = 64. The LSU FU count refers to address adders; data memory has one non-pipelined port.

Initial nonzero registers: R1=12, R2=32, F20=3.
Initial nonzero memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4.
All omitted registers and memory words start at zero; R0 remains zero.

List of instructions

Byte PC / instruction	Byte PC / instruction
`0 Ld F2 0(R1)`	`20 Addi R1 R1 -4`
`4 Mult.d F4 F2 F20`	`24 Addi R2 R2 -4`
`8 Ld F6 0(R2)`	`28 Bne R1 R0 -8`
`12 Add.d F6 F4 F6`	`32 Add.d F20 F2 F2`
`16 Sd F6 0(R2)`	

Results at each issue width

Width	Cycles	Commit	IPC	Speedup	Mispred.	Replay	Fwd.	Squash
1	66	25	0.3788	1.000x	2	0	0	1
2	65	25	0.3846	1.015x	2	0	0	1
3	65	25	0.3846	1.015x	2	0	0	1
4	65	25	0.3846	1.015x	2	0	0	1

IPC = committed instructions / cycles; speedup = width-1 cycles / current cycles. Mispred. counts mispredictions, Replay counts memory replays, Fwd. counts forwarded loads, and Squash counts discarded dynamic instructions.

Verified final state, identical at all four widths

Nonzero registers: R2=20, F2=3, F4=9, F6=15, F20=6.
Nonzero memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=15, Mem[28]=11, Mem[32]=7.
All omitted registers and memory words are zero. I compared all 64 registers and all 64 memory words with the independent sequential interpreter; all four comparisons passed.

I measured 66, 65, 65, 65 cycles at widths 1, 2, 3, and 4. The loop keeps its dependencies and memory traffic at every width, so the wider issue stage produces only a small change in total time. I verified the three loop iterations and the final store values against the sequential interpreter.

4.1 Corrected starter loop - issue width 1

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. Repeated PCs identify loop iterations; their distinct IDs prevent different iterations from being confused.

Issue width 1

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Ld F2 0(R1)	1	2	3-7	8	9
2	4	Mult.d F4 F2 F20	2	9-23	-	24	25
3	8	Ld F6 0(R2)	3	4	8-12	13	26
4	12	Add.d F6 F4 F6	4	25-28	-	29	30
5	16	Sd F6 0(R2)	5	6	34-38	-	38
6	20	Addi R1 R1 -4	6	7	-	9	39
7	24	Addi R2 R2 -4	7	8	-	10	40
8	28	Bne R1 R0 -8	8	10	-	-	41
10	0	Ld F2 0(R1)	12	13	14-18	19	42
11	4	Mult.d F4 F2 F20	13	20-34	-	35	43
12	8	Ld F6 0(R2)	14	15	19-23	25	44
13	12	Add.d F6 F4 F6	15	36-39	-	40	45
14	16	Sd F6 0(R2)	16	17	46-50	-	50
15	20	Addi R1 R1 -4	17	18	-	20	51
16	24	Addi R2 R2 -4	18	19	-	21	52
17	28	Bne R1 R0 -8	19	21	-	-	53
18	0	Ld F2 0(R1)	20	21	24-28	30	54
19	4	Mult.d F4 F2 F20	21	31-45	-	46	55
20	8	Ld F6 0(R2)	22	23	29-33	34	56
21	12	Add.d F6 F4 F6	23	47-50	-	51	57
22	16	Sd F6 0(R2)	24	25	58-62	-	62
23	20	Addi R1 R1 -4	25	26	-	27	63
24	24	Addi R2 R2 -4	26	27	-	28	64
25	28	Bne R1 R0 -8	27	28	-	-	65
26	32	Add.d F20 F2 F2	30	31-34	-	36	66

Table 4.1-1. 25 committed instructions; 66 total cycles. Source: results/corrected_starter_w1.json.

I measured 66, 65, 65, 65 cycles at widths 1, 2, 3, and 4. The loop keeps its dependencies and memory traffic at every width, so the wider issue stage produces only a small change in total time. I verified the three loop iterations and the final store values against the sequential interpreter.

4.1 Corrected starter loop - issue width 2

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. Repeated PCs identify loop iterations; their distinct IDs prevent different iterations from being confused.

Issue width 2

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Ld F2 0(R1)	1	2	3-7	8	9
2	4	Mult.d F4 F2 F20	1	9-23	-	24	25
3	8	Ld F6 0(R2)	2	3	8-12	13	26
4	12	Add.d F6 F4 F6	2	25-28	-	29	30
5	16	Sd F6 0(R2)	3	4	33-37	-	37
6	20	Addi R1 R1 -4	3	4	-	5	38
7	24	Addi R2 R2 -4	4	5	-	6	39
8	28	Bne R1 R0 -8	4	6	-	-	40
10	0	Ld F2 0(R1)	8	9	13-17	18	41
11	4	Mult.d F4 F2 F20	8	19-33	-	35	42
12	8	Ld F6 0(R2)	9	10	18-22	23	43
13	12	Add.d F6 F4 F6	9	36-39	-	40	44
14	16	Sd F6 0(R2)	10	11	45-49	-	49
15	20	Addi R1 R1 -4	10	11	-	12	50
16	24	Addi R2 R2 -4	11	12	-	14	51
17	28	Bne R1 R0 -8	11	13	-	-	52
18	0	Ld F2 0(R1)	13	14	23-27	28	53
19	4	Mult.d F4 F2 F20	13	29-43	-	44	54
20	8	Ld F6 0(R2)	18	19	28-32	33	55
21	12	Add.d F6 F4 F6	18	45-48	-	49	56
22	16	Sd F6 0(R2)	23	24	57-61	-	61
23	20	Addi R1 R1 -4	23	24	-	25	62
24	24	Addi R2 R2 -4	24	25	-	26	63
25	28	Bne R1 R0 -8	24	26	-	-	64
26	32	Add.d F20 F2 F2	28	29-32	-	34	65

Table 4.1-2. 25 committed instructions; 65 total cycles. Source: results/corrected_starter_w2.json.

I measured 66, 65, 65, 65 cycles at widths 1, 2, 3, and 4. The loop keeps its dependencies and memory traffic at every width, so the wider issue stage produces only a small change in total time. I verified the three loop iterations and the final store values against the sequential interpreter.

4.1 Corrected starter loop - issue width 3

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. Repeated PCs identify loop iterations; their distinct IDs prevent different iterations from being confused.

Issue width 3

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Ld F2 0(R1)	1	2	3-7	8	9
2	4	Mult.d F4 F2 F20	1	9-23	-	24	25
3	8	Ld F6 0(R2)	1	3	8-12	13	26
4	12	Add.d F6 F4 F6	2	25-28	-	29	30
5	16	Sd F6 0(R2)	2	4	33-37	-	37
6	20	Addi R1 R1 -4	2	3	-	4	38
7	24	Addi R2 R2 -4	3	4	-	5	39
8	28	Bne R1 R0 -8	3	5	-	-	40
10	0	Ld F2 0(R1)	7	8	13-17	18	41
11	4	Mult.d F4 F2 F20	7	19-33	-	35	42
12	8	Ld F6 0(R2)	7	9	18-22	23	43
13	12	Add.d F6 F4 F6	8	36-39	-	40	44
14	16	Sd F6 0(R2)	8	10	45-49	-	49
15	20	Addi R1 R1 -4	8	9	-	10	50
16	24	Addi R2 R2 -4	9	10	-	11	51
17	28	Bne R1 R0 -8	9	11	-	-	52
18	0	Ld F2 0(R1)	13	14	23-27	28	53
19	4	Mult.d F4 F2 F20	13	29-43	-	44	54
20	8	Ld F6 0(R2)	18	19	28-32	33	55
21	12	Add.d F6 F4 F6	18	45-48	-	49	56
22	16	Sd F6 0(R2)	23	24	57-61	-	61
23	20	Addi R1 R1 -4	23	24	-	25	62
24	24	Addi R2 R2 -4	23	25	-	26	63
25	28	Bne R1 R0 -8	24	26	-	-	64
26	32	Add.d F20 F2 F2	28	29-32	-	34	65

Table 4.1-3. 25 committed instructions; 65 total cycles. Source: results/corrected_starter_w3.json.

I measured 66, 65, 65, 65 cycles at widths 1, 2, 3, and 4. The loop keeps its dependencies and memory traffic at every width, so the wider issue stage produces only a small change in total time. I verified the three loop iterations and the final store values against the sequential interpreter.

4.1 Corrected starter loop - issue width 4

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. Repeated PCs identify loop iterations; their distinct IDs prevent different iterations from being confused.

Issue width 4

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Ld F2 0(R1)	1	2	3-7	8	9
2	4	Mult.d F4 F2 F20	1	9-23	-	24	25
3	8	Ld F6 0(R2)	1	3	8-12	13	26
4	12	Add.d F6 F4 F6	1	25-28	-	29	30
5	16	Sd F6 0(R2)	2	4	33-37	-	37
6	20	Addi R1 R1 -4	2	3	-	4	38
7	24	Addi R2 R2 -4	2	4	-	5	39
8	28	Bne R1 R0 -8	2	5	-	-	40
10	0	Ld F2 0(R1)	7	8	13-17	18	41
11	4	Mult.d F4 F2 F20	7	19-33	-	35	42
12	8	Ld F6 0(R2)	7	9	18-22	23	43
13	12	Add.d F6 F4 F6	7	36-39	-	40	44
14	16	Sd F6 0(R2)	8	10	45-49	-	49
15	20	Addi R1 R1 -4	8	9	-	10	50
16	24	Addi R2 R2 -4	8	10	-	11	51
17	28	Bne R1 R0 -8	8	11	-	-	52
18	0	Ld F2 0(R1)	13	14	23-27	28	53
19	4	Mult.d F4 F2 F20	13	29-43	-	44	54
20	8	Ld F6 0(R2)	18	19	28-32	33	55
21	12	Add.d F6 F4 F6	18	45-48	-	49	56
22	16	Sd F6 0(R2)	23	24	57-61	-	61
23	20	Addi R1 R1 -4	23	24	-	25	62
24	24	Addi R2 R2 -4	23	25	-	26	63
25	28	Bne R1 R0 -8	23	26	-	-	64
26	32	Add.d F20 F2 F2	28	29-32	-	34	65

Table 4.1-4. 25 committed instructions; 65 total cycles. Source: results/corrected_starter_w4.json.

I measured 66, 65, 65, 65 cycles at widths 1, 2, 3, and 4. The loop keeps its dependencies and memory traffic at every width, so the wider issue stage produces only a small change in total time. I verified the three loop iterations and the final store values against the sequential interpreter.

4.2 Independent mixed operations

I spread independent arithmetic and loads across the available units. I used this case to check how a wider issue stage interacts with a single CDB, one memory port, and the available execution units.

Input: benchmarks/mixed_independent.in. I kept the hardware fixed for widths 1-4.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	12	1	-	1
FP adder	12	4	-	1
FP multiplier	12	16	-	1
Load/store address unit	12	1	3	1

ROB entries = 64. The LSU FU count refers to address adders; data memory has one non-pipelined port.

Initial nonzero registers: R2=3, R3=7, F20=1.5, F21=2, F22=4, F23=8.
Initial nonzero memory: Mem[0]=10, Mem[4]=20, Mem[8]=30, Mem[12]=40, Mem[16]=50, Mem[20]=60, Mem[24]=70.
All omitted registers and memory words start at zero; R0 remains zero.

List of instructions

Byte PC / instruction	Byte PC / instruction
`0 Add R10 R2 R3`	`56 Mult.d F11 F20 F23`
`4 Add.d F1 F20 F21`	`60 Ld F12 12(R1)`
`8 Mult.d F2 F20 F22`	`64 Addi R14 R3 -2`
`12 Ld F3 0(R1)`	`68 Sub.d F13 F22 F21`
`16 Addi R11 R2 1`	`72 Mult.d F14 F21 F22`
`20 Add.d F4 F21 F22`	`76 Ld F15 16(R1)`
`24 Mult.d F5 F21 F23`	`80 Add R15 R2 R2`
`28 Ld F6 4(R1)`	`84 Add.d F16 F20 F23`
`32 Sub R12 R3 R2`	`88 Mult.d F17 F20 F21`
`36 Sub.d F7 F23 F20`	`92 Ld F18 20(R1)`
`40 Mult.d F8 F22 F23`	`96 Sub R16 R2 R3`
`44 Ld F9 8(R1)`	`100 Add.d F19 F21 F23`
`48 Add R13 R3 R3`	`104 Mult.d F24 F20 F20`
`52 Add.d F10 F22 F23`	`108 Ld F25 24(R1)`

Results at each issue width

Width	Cycles	Commit	IPC	Speedup	Mispred.	Replay	Fwd.	Squash
1	47	28	0.5957	1.000x	0	0	0	0
2	48	28	0.5833	0.979x	0	0	0	0
3	47	28	0.5957	1.000x	0	0	0	0
4	47	28	0.5957	1.000x	0	0	0	0

IPC = committed instructions / cycles; speedup = width-1 cycles / current cycles. Mispred. counts mispredictions, Replay counts memory replays, Fwd. counts forwarded loads, and Squash counts discarded dynamic instructions.

Verified final state, identical at all four widths

Nonzero registers: R2=3, R3=7, R10=10, R11=4, R12=4, R13=14, R14=5, R15=6, R16=-4, F1=3.5, F2=6, F3=10, F4=6, F5=16, F6=20, F7=6.5, F8=32, F9=30, F10=12, F11=12, F12=40, F13=2, F14=8, F15=50, F16=9.5, F17=3, F18=60, F19=10, F20=1.5, F21=2, F22=4, F23=8, F24=2.25, F25=70.
Nonzero memory: Mem[0]=10, Mem[4]=20, Mem[8]=30, Mem[12]=40, Mem[16]=50, Mem[20]=60, Mem[24]=70.
All omitted registers and memory words are zero. I compared all 64 registers and all 64 memory words with the independent sequential interpreter; all four comparisons passed.

I measured 47, 48, 47, 47 cycles at widths 1, 2, 3, and 4. Earlier issue does not remove contention for the single CDB or memory port. The width-2 run is slightly slower in this schedule; I therefore do not treat a larger issue width as a guarantee of a speedup.

4.2 Independent mixed operations - issue width 1

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 1

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R10 R2 R3	1	2	-	3	4
2	4	Add.d F1 F20 F21	2	3-6	-	7	8
3	8	Mult.d F2 F20 F22	3	4-19	-	21	22
4	12	Ld F3 0(R1)	4	5	6-8	9	23
5	16	Addi R11 R2 1	5	6	-	8	24
6	20	Add.d F4 F21 F22	6	7-10	-	11	25
7	24	Mult.d F5 F21 F23	7	8-23	-	25	26
8	28	Ld F6 4(R1)	8	9	10-12	13	27
9	32	Sub R12 R3 R2	9	10	-	12	28
10	36	Sub.d F7 F23 F20	10	11-14	-	15	29
11	40	Mult.d F8 F22 F23	11	12-27	-	29	30
12	44	Ld F9 8(R1)	12	13	14-16	17	31
13	48	Add R13 R3 R3	13	14	-	16	32
14	52	Add.d F10 F22 F23	14	15-18	-	19	33
15	56	Mult.d F11 F20 F23	15	16-31	-	32	34
16	60	Ld F12 12(R1)	16	17	18-20	22	35
17	64	Addi R14 R3 -2	17	18	-	20	36
18	68	Sub.d F13 F22 F21	18	19-22	-	23	37
19	72	Mult.d F14 F21 F22	19	20-35	-	36	38
20	76	Ld F15 16(R1)	20	21	22-24	26	39
21	80	Add R15 R2 R2	21	22	-	24	40
22	84	Add.d F16 F20 F23	22	23-26	-	27	41
23	88	Mult.d F17 F20 F21	23	24-39	-	40	42
24	92	Ld F18 20(R1)	24	25	26-28	30	43
25	96	Sub R16 R2 R3	25	26	-	28	44
26	100	Add.d F19 F21 F23	26	27-30	-	31	45
27	104	Mult.d F24 F20 F20	27	28-43	-	44	46
28	108	Ld F25 24(R1)	28	29	30-32	33	47

Table 4.2-1. 28 committed instructions; 47 total cycles. Source: results/mixed_independent_w1.json.

I measured 47, 48, 47, 47 cycles at widths 1, 2, 3, and 4. Earlier issue does not remove contention for the single CDB or memory port. The width-2 run is slightly slower in this schedule; I therefore do not treat a larger issue width as a guarantee of a speedup.

4.2 Independent mixed operations - issue width 2

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 2

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R10 R2 R3	1	2	-	3	4
2	4	Add.d F1 F20 F21	1	2-5	-	6	7
3	8	Mult.d F2 F20 F22	2	3-18	-	22	23
4	12	Ld F3 0(R1)	2	3	4-6	7	24
5	16	Addi R11 R2 1	3	4	-	5	25
6	20	Add.d F4 F21 F22	3	4-7	-	9	26
7	24	Mult.d F5 F21 F23	4	5-20	-	24	27
8	28	Ld F6 4(R1)	4	5	7-9	11	28
9	32	Sub R12 R3 R2	5	6	-	8	29
10	36	Sub.d F7 F23 F20	5	6-9	-	12	30
11	40	Mult.d F8 F22 F23	6	7-22	-	26	31
12	44	Ld F9 8(R1)	6	7	10-12	15	32
13	48	Add R13 R3 R3	7	8	-	10	33
14	52	Add.d F10 F22 F23	7	8-11	-	14	34
15	56	Mult.d F11 F20 F23	8	9-24	-	27	35
16	60	Ld F12 12(R1)	8	9	13-15	19	36
17	64	Addi R14 R3 -2	9	10	-	13	37
18	68	Sub.d F13 F22 F21	9	10-13	-	17	38
19	72	Mult.d F14 F21 F22	10	11-26	-	29	39
20	76	Ld F15 16(R1)	10	11	16-18	23	40
21	80	Add R15 R2 R2	11	12	-	16	41
22	84	Add.d F16 F20 F23	11	12-15	-	20	42
23	88	Mult.d F17 F20 F21	12	13-28	-	30	43
24	92	Ld F18 20(R1)	12	13	19-21	25	44
25	96	Sub R16 R2 R3	13	14	-	18	45
26	100	Add.d F19 F21 F23	13	14-17	-	21	46
27	104	Mult.d F24 F20 F20	14	15-30	-	31	47
28	108	Ld F25 24(R1)	14	15	22-24	28	48

Table 4.2-2. 28 committed instructions; 48 total cycles. Source: results/mixed_independent_w2.json.

I measured 47, 48, 47, 47 cycles at widths 1, 2, 3, and 4. Earlier issue does not remove contention for the single CDB or memory port. The width-2 run is slightly slower in this schedule; I therefore do not treat a larger issue width as a guarantee of a speedup.

4.2 Independent mixed operations - issue width 3

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 3

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R10 R2 R3	1	2	-	3	4
2	4	Add.d F1 F20 F21	1	2-5	-	6	7
3	8	Mult.d F2 F20 F22	1	2-17	-	21	22
4	12	Ld F3 0(R1)	2	3	4-6	7	23
5	16	Addi R11 R2 1	2	3	-	4	24
6	20	Add.d F4 F21 F22	2	3-6	-	8	25
7	24	Mult.d F5 F21 F23	3	4-19	-	23	26
8	28	Ld F6 4(R1)	3	4	7-9	13	27
9	32	Sub R12 R3 R2	3	4	-	5	28
10	36	Sub.d F7 F23 F20	4	5-8	-	11	29
11	40	Mult.d F8 F22 F23	4	5-20	-	24	30
12	44	Ld F9 8(R1)	4	5	10-12	17	31
13	48	Add R13 R3 R3	5	6	-	9	32
14	52	Add.d F10 F22 F23	5	6-9	-	14	33
15	56	Mult.d F11 F20 F23	5	6-21	-	25	34
16	60	Ld F12 12(R1)	6	7	13-15	20	35
17	64	Addi R14 R3 -2	6	7	-	10	36
18	68	Sub.d F13 F22 F21	6	7-10	-	15	37
19	72	Mult.d F14 F21 F22	7	8-23	-	27	38
20	76	Ld F15 16(R1)	7	8	16-18	22	39
21	80	Add R15 R2 R2	7	8	-	12	40
22	84	Add.d F16 F20 F23	8	9-12	-	18	41
23	88	Mult.d F17 F20 F21	8	9-24	-	28	42
24	92	Ld F18 20(R1)	8	9	19-21	26	43
25	96	Sub R16 R2 R3	9	10	-	16	44
26	100	Add.d F19 F21 F23	9	10-13	-	19	45
27	104	Mult.d F24 F20 F20	9	10-25	-	30	46
28	108	Ld F25 24(R1)	10	11	22-24	29	47

Table 4.2-3. 28 committed instructions; 47 total cycles. Source: results/mixed_independent_w3.json.

I measured 47, 48, 47, 47 cycles at widths 1, 2, 3, and 4. Earlier issue does not remove contention for the single CDB or memory port. The width-2 run is slightly slower in this schedule; I therefore do not treat a larger issue width as a guarantee of a speedup.

4.2 Independent mixed operations - issue width 4

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 4

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R10 R2 R3	1	2	-	3	4
2	4	Add.d F1 F20 F21	1	2-5	-	6	7
3	8	Mult.d F2 F20 F22	1	2-17	-	21	22
4	12	Ld F3 0(R1)	1	2	3-5	7	23
5	16	Addi R11 R2 1	2	3	-	4	24
6	20	Add.d F4 F21 F22	2	3-6	-	9	25
7	24	Mult.d F5 F21 F23	2	3-18	-	23	26
8	28	Ld F6 4(R1)	2	3	6-8	13	27
9	32	Sub R12 R3 R2	3	4	-	5	28
10	36	Sub.d F7 F23 F20	3	4-7	-	11	29
11	40	Mult.d F8 F22 F23	3	4-19	-	24	30
12	44	Ld F9 8(R1)	3	4	9-11	18	31
13	48	Add R13 R3 R3	4	5	-	8	32
14	52	Add.d F10 F22 F23	4	5-8	-	14	33
15	56	Mult.d F11 F20 F23	4	5-20	-	25	34
16	60	Ld F12 12(R1)	4	5	12-14	20	35
17	64	Addi R14 R3 -2	5	6	-	10	36
18	68	Sub.d F13 F22 F21	5	6-9	-	16	37
19	72	Mult.d F14 F21 F22	5	6-21	-	27	38
20	76	Ld F15 16(R1)	5	6	15-17	22	39
21	80	Add R15 R2 R2	6	7	-	12	40
22	84	Add.d F16 F20 F23	6	7-10	-	17	41
23	88	Mult.d F17 F20 F21	6	7-22	-	28	42
24	92	Ld F18 20(R1)	6	7	18-20	26	43
25	96	Sub R16 R2 R3	7	8	-	15	44
26	100	Add.d F19 F21 F23	7	8-11	-	19	45
27	104	Mult.d F24 F20 F20	7	8-23	-	29	46
28	108	Ld F25 24(R1)	7	8	21-23	30	47

Table 4.2-4. 28 committed instructions; 47 total cycles. Source: results/mixed_independent_w4.json.

I measured 47, 48, 47, 47 cycles at widths 1, 2, 3, and 4. Earlier issue does not remove contention for the single CDB or memory port. The width-2 run is slightly slower in this schedule; I therefore do not treat a larger issue width as a guarantee of a speedup.

4.3 Dependencies and register renaming

I used repeated destinations and dependent integer/FP operations to check RAW, WAR, and WAW handling. Later instructions must use the correct producer even when several instructions issue together.

Input: benchmarks/dependency_chain.in. I kept the hardware fixed for widths 1-4.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	6	2	-	1
FP adder	5	3	-	1
FP multiplier	3	6	-	1
Load/store address unit	3	1	4	1

ROB entries = 32. The LSU FU count refers to address adders; data memory has one non-pipelined port.

Initial nonzero registers: R1=5, R2=3, F1=2, F2=3.
Initial nonzero memory: none.
All omitted registers and memory words start at zero; R0 remains zero.

List of instructions

Byte PC / instruction	Byte PC / instruction
`0 Add R3 R1 R2`	`28 Mult.d F4 F3 F2`
`4 Addi R1 R3 1`	`32 Add.d F1 F4 F3`
`8 Sub R4 R1 R2`	`36 Sub.d F3 F1 F2`
`12 Add R3 R4 R1`	`40 Mult.d F5 F3 F1`
`16 Addi R5 R3 -2`	`44 Add.d F6 F5 F4`
`20 Add R6 R5 R4`	`48 Addi R3 R6 1`
`24 Add.d F3 F1 F2`	`52 Add R7 R3 R1`

Results at each issue width

Width	Cycles	Commit	IPC	Speedup	Mispred.	Replay	Fwd.	Squash
1	40	14	0.3500	1.000x	0	0	0	0
2	38	14	0.3684	1.053x	0	0	0	0
3	38	14	0.3684	1.053x	0	0	0	0
4	36	14	0.3889	1.111x	0	0	0	0

IPC = committed instructions / cycles; speedup = width-1 cycles / current cycles. Mispred. counts mispredictions, Replay counts memory replays, Fwd. counts forwarded loads, and Squash counts discarded dynamic instructions.

Verified final state, identical at all four widths

Nonzero registers: R1=9, R2=3, R3=20, R4=6, R5=13, R6=19, R7=29, F1=20, F2=3, F3=17, F4=15, F5=340, F6=355.
Nonzero memory: none.
All omitted registers and memory words are zero. I compared all 64 registers and all 64 memory words with the independent sequential interpreter; all four comparisons passed.

I measured 40, 38, 38, 36 cycles at widths 1, 2, 3, and 4. The wider issue stage admits the FP operations sooner, while each dependent chain still waits for its producers. The width-4 result takes fewer cycles even though the dependency and retirement rules are unchanged.

4.3 Dependencies and register renaming - issue widths 1 and 2

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 1

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R3 R1 R2	1	2-3	-	4	5
2	4	Addi R1 R3 1	2	5-6	-	7	8
3	8	Sub R4 R1 R2	3	8-9	-	10	11
4	12	Add R3 R4 R1	4	11-12	-	13	14
5	16	Addi R5 R3 -2	5	14-15	-	16	17
6	20	Add R6 R5 R4	6	17-18	-	19	20
7	24	Add.d F3 F1 F2	7	8-10	-	11	21
8	28	Mult.d F4 F3 F2	8	12-17	-	18	22
9	32	Add.d F1 F4 F3	9	19-21	-	22	23
10	36	Sub.d F3 F1 F2	10	23-25	-	26	27
11	40	Mult.d F5 F3 F1	11	27-32	-	33	34
12	44	Add.d F6 F5 F4	12	34-36	-	37	38
13	48	Addi R3 R6 1	13	20-21	-	23	39
14	52	Add R7 R3 R1	14	24-25	-	27	40

Table 4.3-1. 14 committed instructions; 40 total cycles. Source: results/dependency_chain_w1.json.

Issue width 2

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R3 R1 R2	1	2-3	-	4	5
2	4	Addi R1 R3 1	1	5-6	-	7	8
3	8	Sub R4 R1 R2	2	8-9	-	10	11
4	12	Add R3 R4 R1	2	11-12	-	13	14
5	16	Addi R5 R3 -2	3	14-15	-	16	17
6	20	Add R6 R5 R4	3	17-18	-	19	20
7	24	Add.d F3 F1 F2	4	5-7	-	8	21
8	28	Mult.d F4 F3 F2	4	9-14	-	15	22
9	32	Add.d F1 F4 F3	5	16-18	-	20	23
10	36	Sub.d F3 F1 F2	5	21-23	-	24	25
11	40	Mult.d F5 F3 F1	6	25-30	-	31	32
12	44	Add.d F6 F5 F4	6	32-34	-	35	36
13	48	Addi R3 R6 1	7	20-21	-	22	37
14	52	Add R7 R3 R1	7	23-24	-	25	38

Table 4.3-2. 14 committed instructions; 38 total cycles. Source: results/dependency_chain_w2.json.

4.3 Dependencies and register renaming - issue widths 3 and 4

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 3

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R3 R1 R2	1	2-3	-	4	5
2	4	Addi R1 R3 1	1	5-6	-	7	8
3	8	Sub R4 R1 R2	1	8-9	-	10	11
4	12	Add R3 R4 R1	2	11-12	-	13	14
5	16	Addi R5 R3 -2	2	14-15	-	16	17
6	20	Add R6 R5 R4	2	17-18	-	19	20
7	24	Add.d F3 F1 F2	3	4-6	-	8	21
8	28	Mult.d F4 F3 F2	3	9-14	-	15	22
9	32	Add.d F1 F4 F3	3	16-18	-	20	23
10	36	Sub.d F3 F1 F2	4	21-23	-	24	25
11	40	Mult.d F5 F3 F1	4	25-30	-	31	32
12	44	Add.d F6 F5 F4	4	32-34	-	35	36
13	48	Addi R3 R6 1	5	20-21	-	22	37
14	52	Add R7 R3 R1	6	23-24	-	25	38

Table 4.3-3. 14 committed instructions; 38 total cycles. Source: results/dependency_chain_w3.json.

Issue width 4

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Add R3 R1 R2	1	2-3	-	4	5
2	4	Addi R1 R3 1	1	5-6	-	7	8
3	8	Sub R4 R1 R2	1	8-9	-	10	11
4	12	Add R3 R4 R1	1	11-12	-	13	14
5	16	Addi R5 R3 -2	2	14-15	-	16	17
6	20	Add R6 R5 R4	2	17-18	-	19	20
7	24	Add.d F3 F1 F2	2	3-5	-	6	21
8	28	Mult.d F4 F3 F2	2	7-12	-	14	22
9	32	Add.d F1 F4 F3	3	15-17	-	18	23
10	36	Sub.d F3 F1 F2	3	19-21	-	22	24
11	40	Mult.d F5 F3 F1	3	23-28	-	29	30
12	44	Add.d F6 F5 F4	3	30-32	-	33	34
13	48	Addi R3 R6 1	4	20-21	-	23	35
14	52	Add R7 R3 R1	6	24-25	-	26	36

Table 4.3-4. 14 committed instructions; 36 total cycles. Source: results/dependency_chain_w4.json.

4.4 Store-to-load forwarding

I placed a long multiplication before stores and loads so older stores remain uncommitted while younger loads execute. Two stores target address zero, so the load must use the younger matching store.

Input: benchmarks/forwarding.in. I kept the hardware fixed for widths 1-4.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	4	2	-	1
FP multiplier	2	24	-	1
Load/store address unit	8	1	4	1

ROB entries = 32. The LSU FU count refers to address adders; data memory has one non-pipelined port.

Initial nonzero registers: F1=3, F2=4.
Initial nonzero memory: Mem[0]=99, Mem[4]=10, Mem[8]=11.
All omitted registers and memory words start at zero; R0 remains zero.

List of instructions

Byte PC / instruction	Byte PC / instruction
`0 Mult.d F10 F1 F2`	`20 Sd F4 4(R1)`
`4 Sd F1 0(R1)`	`24 Ld F5 4(R1)`
`8 Sd F2 0(R1)`	`28 Ld F6 8(R1)`
`12 Ld F3 0(R1)`	`32 Ld F7 12(R1)`
`16 Add.d F4 F3 F1`	

Results at each issue width

Width	Cycles	Commit	IPC	Speedup	Mispred.	Replay	Fwd.	Squash
1	44	9	0.2045	1.000x	0	0	2	0
2	44	9	0.2045	1.000x	0	0	2	0
3	44	9	0.2045	1.000x	0	0	2	0
4	44	9	0.2045	1.000x	0	0	2	0

IPC = committed instructions / cycles; speedup = width-1 cycles / current cycles. Mispred. counts mispredictions, Replay counts memory replays, Fwd. counts forwarded loads, and Squash counts discarded dynamic instructions.

Verified final state, identical at all four widths

Nonzero registers: F1=3, F2=4, F3=4, F4=7, F5=7, F6=11, F10=12.
Nonzero memory: Mem[0]=4, Mem[4]=7, Mem[8]=11.
All omitted registers and memory words are zero. I compared all 64 registers and all 64 memory words with the independent sequential interpreter; all four comparisons passed.

I measured 44, 44, 44, 44 cycles at widths 1, 2, 3, and 4. The long multiply delays retirement, and loads can obtain the correct values from older stores before those stores commit. I checked that the load at byte PC 12 reads 4 from the second store to address zero, rather than 3 from the first store or 99 from old memory.

4.4 Store-to-load forwarding - issue widths 1 and 2

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 1

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F10 F1 F2	1	2-25	-	26	27
2	4	Sd F1 0(R1)	2	3	28-31	-	31
3	8	Sd F2 0(R1)	3	4	32-35	-	35
4	12	Ld F3 0(R1)	4	5	6*	7	36
5	16	Add.d F4 F3 F1	5	8-9	-	10	37
6	20	Sd F4 4(R1)	6	7	38-41	-	41
7	24	Ld F5 4(R1)	7	8	14*	15	42
8	28	Ld F6 8(R1)	8	9	10-13	14	43
9	32	Ld F7 12(R1)	9	10	15-18	19	44

Table 4.4-1. 9 committed instructions; 44 total cycles. Source: results/forwarding_w1.json.

Issue width 2

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F10 F1 F2	1	2-25	-	26	27
2	4	Sd F1 0(R1)	1	2	28-31	-	31
3	8	Sd F2 0(R1)	2	3	32-35	-	35
4	12	Ld F3 0(R1)	2	4	5*	6	36
5	16	Add.d F4 F3 F1	3	7-8	-	9	37
6	20	Sd F4 4(R1)	3	5	38-41	-	41
7	24	Ld F5 4(R1)	4	6	12*	13	42
8	28	Ld F6 8(R1)	4	7	8-11	12	43
9	32	Ld F7 12(R1)	5	8	13-16	17	44

Table 4.4-2. 9 committed instructions; 44 total cycles. Source: results/forwarding_w2.json.

4.4 Store-to-load forwarding - issue widths 3 and 4

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 3

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F10 F1 F2	1	2-25	-	26	27
2	4	Sd F1 0(R1)	1	2	28-31	-	31
3	8	Sd F2 0(R1)	1	3	32-35	-	35
4	12	Ld F3 0(R1)	2	4	5*	6	36
5	16	Add.d F4 F3 F1	2	7-8	-	9	37
6	20	Sd F4 4(R1)	2	5	38-41	-	41
7	24	Ld F5 4(R1)	3	6	12*	13	42
8	28	Ld F6 8(R1)	3	7	8-11	12	43
9	32	Ld F7 12(R1)	3	8	13-16	17	44

Table 4.4-3. 9 committed instructions; 44 total cycles. Source: results/forwarding_w3.json.

Issue width 4

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F10 F1 F2	1	2-25	-	26	27
2	4	Sd F1 0(R1)	1	2	28-31	-	31
3	8	Sd F2 0(R1)	1	3	32-35	-	35
4	12	Ld F3 0(R1)	1	4	5*	6	36
5	16	Add.d F4 F3 F1	2	7-8	-	9	37
6	20	Sd F4 4(R1)	2	5	38-41	-	41
7	24	Ld F5 4(R1)	2	6	12*	13	42
8	28	Ld F6 8(R1)	2	7	8-11	12	43
9	32	Ld F7 12(R1)	3	8	13-16	17	44

Table 4.4-4. 9 committed instructions; 44 total cycles. Source: results/forwarding_w4.json.

4.5 Branch prediction and recovery

I used a four-iteration loop followed by a taken branch that skips a store and a register update. I checked that prediction recovery removes speculative work without changing the final program result.

Input: benchmarks/branch_recovery.in. I kept the hardware fixed for widths 1-4.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	12	2	-	1
FP adder	4	3	-	1
FP multiplier	2	8	-	1
Load/store address unit	8	1	3	1

ROB entries = 48. The LSU FU count refers to address adders; data memory has one non-pipelined port.

Initial nonzero registers: R2=4, R3=100, F1=1, F2=2.
Initial nonzero memory: Mem[0]=10, Mem[4]=20.
All omitted registers and memory words start at zero; R0 remains zero.

List of instructions

Byte PC / instruction	Byte PC / instruction
`0 Addi R1 R1 1`	`20 Sd F2 0(R0)`
`4 Addi R3 R3 2`	`24 Addi R3 R3 999`
`8 Bne R1 R2 -3`	`28 Add.d F3 F1 F2`
`12 Add R4 R1 R3`	`32 Sd F3 4(R0)`
`16 Beq R1 R2 2`	`36 Addi R5 R4 1`

Results at each issue width

Width	Cycles	Commit	IPC	Speedup	Mispred.	Replay	Fwd.	Squash
1	44	17	0.3864	1.000x	3	0	0	17
2	44	17	0.3864	1.000x	3	0	0	24
3	44	17	0.3864	1.000x	3	0	0	24
4	44	17	0.3864	1.000x	3	0	0	24

IPC = committed instructions / cycles; speedup = width-1 cycles / current cycles. Mispred. counts mispredictions, Replay counts memory replays, Fwd. counts forwarded loads, and Squash counts discarded dynamic instructions.

Verified final state, identical at all four widths

Nonzero registers: R1=4, R2=4, R3=108, R4=112, R5=113, F1=1, F2=2, F3=3.
Nonzero memory: Mem[0]=10, Mem[4]=3.
All omitted registers and memory words are zero. I compared all 64 registers and all 64 memory words with the independent sequential interpreter; all four comparisons passed.

I measured 44, 44, 44, 44 cycles at widths 1, 2, 3, and 4. The loop and branch dependencies determine the total time. Wider issue creates more wrong-path work in this case. I confirmed that Mem[0] stays 10, the skipped register increment has no effect, and the final store changes Mem[4] to 3.

4.5 Branch prediction and recovery - issue widths 1 and 2

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. Repeated PCs identify loop iterations; their distinct IDs prevent different iterations from being confused.

Issue width 1

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R1 R1 1	1	2-3	-	4	5
2	4	Addi R3 R3 2	2	4-5	-	6	7
3	8	Bne R1 R2 -3	3	6-7	-	-	8
8	0	Addi R1 R1 1	9	10-11	-	12	13
9	4	Addi R3 R3 2	10	12-13	-	14	15
10	8	Bne R1 R2 -3	11	14-15	-	-	16
11	0	Addi R1 R1 1	12	16-17	-	18	19
12	4	Addi R3 R3 2	13	18-19	-	20	21
13	8	Bne R1 R2 -3	14	20-21	-	-	22
14	0	Addi R1 R1 1	15	22-23	-	24	25
15	4	Addi R3 R3 2	16	24-25	-	26	27
16	8	Bne R1 R2 -3	17	26-27	-	-	28
27	12	Add R4 R1 R3	29	30-31	-	32	33
28	16	Beq R1 R2 2	30	32-33	-	-	34
32	28	Add.d F3 F1 F2	35	36-38	-	39	40
33	32	Sd F3 4(R0)	36	37	41-43	-	43
34	36	Addi R5 R4 1	37	38-39	-	40	44

Table 4.5-1. 17 committed instructions; 44 total cycles. Source: results/branch_recovery_w1.json.

Issue width 2

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R1 R1 1	1	2-3	-	4	5
2	4	Addi R3 R3 2	1	4-5	-	6	7
3	8	Bne R1 R2 -3	2	6-7	-	-	8
11	0	Addi R1 R1 1	9	10-11	-	12	13
12	4	Addi R3 R3 2	9	12-13	-	14	15
13	8	Bne R1 R2 -3	10	14-15	-	-	16
14	0	Addi R1 R1 1	10	16-17	-	18	19
15	4	Addi R3 R3 2	11	18-19	-	20	21
16	8	Bne R1 R2 -3	11	20-21	-	-	22
17	0	Addi R1 R1 1	12	22-23	-	24	25
18	4	Addi R3 R3 2	12	24-25	-	26	27
19	8	Bne R1 R2 -3	13	26-27	-	-	28
32	12	Add R4 R1 R3	29	30-31	-	32	33
33	16	Beq R1 R2 2	29	32-33	-	-	34
39	28	Add.d F3 F1 F2	35	36-38	-	39	40
40	32	Sd F3 4(R0)	35	36	41-43	-	43
41	36	Addi R5 R4 1	36	37-38	-	40	44

Table 4.5-2. 17 committed instructions; 44 total cycles. Source: results/branch_recovery_w2.json.

4.5 Branch prediction and recovery - issue widths 3 and 4

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. Repeated PCs identify loop iterations; their distinct IDs prevent different iterations from being confused.

Issue width 3

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R1 R1 1	1	2-3	-	4	5
2	4	Addi R3 R3 2	1	4-5	-	6	7
3	8	Bne R1 R2 -3	1	6-7	-	-	8
11	0	Addi R1 R1 1	9	10-11	-	12	13
12	4	Addi R3 R3 2	9	12-13	-	14	15
13	8	Bne R1 R2 -3	9	14-15	-	-	16
14	0	Addi R1 R1 1	10	16-17	-	18	19
15	4	Addi R3 R3 2	10	18-19	-	20	21
16	8	Bne R1 R2 -3	10	20-21	-	-	22
17	0	Addi R1 R1 1	11	22-23	-	24	25
18	4	Addi R3 R3 2	11	24-25	-	26	27
19	8	Bne R1 R2 -3	11	26-27	-	-	28
32	12	Add R4 R1 R3	29	30-31	-	32	33
33	16	Beq R1 R2 2	29	32-33	-	-	34
39	28	Add.d F3 F1 F2	35	36-38	-	39	40
40	32	Sd F3 4(R0)	35	36	41-43	-	43
41	36	Addi R5 R4 1	35	36-37	-	38	44

Table 4.5-3. 17 committed instructions; 44 total cycles. Source: results/branch_recovery_w3.json.

Issue width 4

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R1 R1 1	1	2-3	-	4	5
2	4	Addi R3 R3 2	1	4-5	-	6	7
3	8	Bne R1 R2 -3	1	6-7	-	-	8
11	0	Addi R1 R1 1	9	10-11	-	12	13
12	4	Addi R3 R3 2	9	12-13	-	14	15
13	8	Bne R1 R2 -3	9	14-15	-	-	16
14	0	Addi R1 R1 1	9	16-17	-	18	19
15	4	Addi R3 R3 2	10	18-19	-	20	21
16	8	Bne R1 R2 -3	10	20-21	-	-	22
17	0	Addi R1 R1 1	10	22-23	-	24	25
18	4	Addi R3 R3 2	10	24-25	-	26	27
19	8	Bne R1 R2 -3	11	26-27	-	-	28
32	12	Add R4 R1 R3	29	30-31	-	32	33
33	16	Beq R1 R2 2	29	32-33	-	-	34
39	28	Add.d F3 F1 F2	35	36-38	-	39	40
40	32	Sd F3 4(R0)	35	36	41-43	-	43
41	36	Addi R5 R4 1	35	36-37	-	38	44

Table 4.5-4. 17 committed instructions; 44 total cycles. Source: results/branch_recovery_w4.json.

4.6 Memory disambiguation and replay

I delayed an older store address with an integer dependency chain. A younger load can read first, so resolving the conflicting store must replay that load and its successors.

Input: benchmarks/memory_replay.in. I kept the hardware fixed for widths 1-4.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	8	4	-	1
FP adder	4	2	-	1
FP multiplier	2	8	-	1
Load/store address unit	10	1	2	1

ROB entries = 32. The LSU FU count refers to address adders; data memory has one non-pipelined port.

Initial nonzero registers: F1=9.
Initial nonzero memory: Mem[16]=1, Mem[20]=2, Mem[24]=3.
All omitted registers and memory words start at zero; R0 remains zero.

List of instructions

Byte PC / instruction	Byte PC / instruction
`0 Addi R10 R0 4`	`20 Add.d F3 F2 F1`
`4 Add R11 R10 R10`	`24 Sd F3 20(R0)`
`8 Add R12 R11 R11`	`28 Ld F4 24(R0)`
`12 Sd F1 0(R12)`	`32 Addi R1 R0 7`
`16 Ld F2 16(R0)`	

Results at each issue width

Width	Cycles	Commit	IPC	Speedup	Mispred.	Replay	Fwd.	Squash
1	34	9	0.2647	1.000x	0	1	0	5
2	34	9	0.2647	1.000x	0	1	0	5
3	35	9	0.2571	0.971x	0	1	0	5
4	35	9	0.2571	0.971x	0	1	0	5

IPC = committed instructions / cycles; speedup = width-1 cycles / current cycles. Mispred. counts mispredictions, Replay counts memory replays, Fwd. counts forwarded loads, and Squash counts discarded dynamic instructions.

Verified final state, identical at all four widths

Nonzero registers: R1=7, R10=4, R11=8, R12=16, F1=9, F2=9, F3=18, F4=3.
Nonzero memory: Mem[16]=9, Mem[20]=18, Mem[24]=3.
All omitted registers and memory words are zero. I compared all 64 registers and all 64 memory words with the independent sequential interpreter; all four comparisons passed.

I measured 34, 34, 35, 35 cycles at widths 1, 2, 3, and 4. Every run detects a memory-order violation and retries the load. The final value F2=9 shows that the stale initial Mem[16]=1 did not survive. The wider runs take one extra cycle in this schedule because earlier speculative work does not eliminate recovery.

4.6 Memory disambiguation and replay - issue widths 1 and 2

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. The first, squashed load attempt is retained in the JSON/text evidence; this table shows the successful committed attempt.

Issue width 1

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R10 R0 4	1	2-5	-	6	7
2	4	Add R11 R10 R10	2	7-10	-	11	12
3	8	Add R12 R11 R11	3	15-18	-	19	20
4	12	Sd F1 0(R12)	4	20	21-22	-	22
10	16	Ld F2 16(R0)	22	23	24-25	26	27
11	20	Add.d F3 F2 F1	23	27-28	-	29	30
12	24	Sd F3 20(R0)	24	25	31-32	-	32
13	28	Ld F4 24(R0)	25	26	27-28	30	33
14	32	Addi R1 R0 7	26	27-30	-	31	34

Table 4.6-1. 9 committed instructions; 34 total cycles. Source: results/memory_replay_w1.json.

Issue width 2

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R10 R0 4	1	2-5	-	6	7
2	4	Add R11 R10 R10	1	10-13	-	14	15
3	8	Add R12 R11 R11	2	15-18	-	19	20
4	12	Sd F1 0(R12)	2	20	21-22	-	22
10	16	Ld F2 16(R0)	22	23	24-25	26	27
11	20	Add.d F3 F2 F1	22	27-28	-	29	30
12	24	Sd F3 20(R0)	23	24	31-32	-	32
13	28	Ld F4 24(R0)	23	25	26-27	28	33
14	32	Addi R1 R0 7	24	25-28	-	30	34

Table 4.6-2. 9 committed instructions; 34 total cycles. Source: results/memory_replay_w2.json.

4.6 Memory disambiguation and replay - issue widths 3 and 4

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB. The first, squashed load attempt is retained in the JSON/text evidence; this table shows the successful committed attempt.

Issue width 3

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R10 R0 4	1	2-5	-	6	7
2	4	Add R11 R10 R10	1	10-13	-	14	15
3	8	Add R12 R11 R11	1	15-18	-	19	20
4	12	Sd F1 0(R12)	2	20	21-22	-	22
10	16	Ld F2 16(R0)	22	23	24-25	26	27
11	20	Add.d F3 F2 F1	22	27-28	-	30	31
12	24	Sd F3 20(R0)	22	24	32-33	-	33
13	28	Ld F4 24(R0)	23	25	26-27	28	34
14	32	Addi R1 R0 7	23	24-27	-	29	35

Table 4.6-3. 9 committed instructions; 35 total cycles. Source: results/memory_replay_w3.json.

Issue width 4

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Addi R10 R0 4	1	2-5	-	6	7
2	4	Add R11 R10 R10	1	10-13	-	14	15
3	8	Add R12 R11 R11	1	15-18	-	19	20
4	12	Sd F1 0(R12)	1	20	21-22	-	22
10	16	Ld F2 16(R0)	22	23	24-25	26	27
11	20	Add.d F3 F2 F1	22	27-28	-	30	31
12	24	Sd F3 20(R0)	22	24	32-33	-	33
13	28	Ld F4 24(R0)	22	25	26-27	28	34
14	32	Addi R1 R0 7	23	24-27	-	29	35

Table 4.6-4. 9 committed instructions; 35 total cycles. Source: results/memory_replay_w4.json.

4.7 Small ROB, reservation stations, and LSQ

I reduced the ROB, reservation-station banks, and load/store queue to force structural stalls. I checked that issue stops at the first blocked instruction and that the program still completes correctly.

Input: benchmarks/resource_pressure.in. I kept the hardware fixed for widths 1-4.

Unit	RS / queue	EX cycles	MEM cycles	FUs
Integer adder	1	2	-	1
FP adder	1	3	-	1
FP multiplier	1	8	-	1
Load/store address unit	2	1	3	1

ROB entries = 4. The LSU FU count refers to address adders; data memory has one non-pipelined port.

Initial nonzero registers: R2=1, R3=2, F1=2, F2=3.
Initial nonzero memory: Mem[0]=1.
All omitted registers and memory words start at zero; R0 remains zero.

List of instructions

Byte PC / instruction	Byte PC / instruction
`0 Mult.d F3 F1 F2`	`28 Ld F7 4(R1)`
`4 Add.d F4 F3 F1`	`32 Mult.d F8 F7 F2`
`8 Add R4 R2 R3`	`36 Addi R6 R5 1`
`12 Addi R5 R4 1`	`40 Add.d F9 F8 F1`
`16 Ld F5 0(R1)`	`44 Sd F9 8(R1)`
`20 Add.d F6 F5 F4`	`48 Ld F10 8(R1)`
`24 Sd F6 4(R1)`	`52 Sub R7 R6 R2`

Results at each issue width

Width	Cycles	Commit	IPC	Speedup	Mispred.	Replay	Fwd.	Squash
1	42	14	0.3333	1.000x	0	0	2	0
2	42	14	0.3333	1.000x	0	0	2	0
3	42	14	0.3333	1.000x	0	0	2	0
4	42	14	0.3333	1.000x	0	0	2	0

IPC = committed instructions / cycles; speedup = width-1 cycles / current cycles. Mispred. counts mispredictions, Replay counts memory replays, Fwd. counts forwarded loads, and Squash counts discarded dynamic instructions.

Verified final state, identical at all four widths

Nonzero registers: R2=1, R3=2, R4=3, R5=4, R6=5, R7=4, F1=2, F2=3, F3=6, F4=8, F5=1, F6=9, F7=9, F8=27, F9=29, F10=29.
Nonzero memory: Mem[0]=1, Mem[4]=9, Mem[8]=29.
All omitted registers and memory words are zero. I compared all 64 registers and all 64 memory words with the independent sequential interpreter; all four comparisons passed.

I measured 42, 42, 42, 42 cycles at widths 1, 2, 3, and 4. The small queues and four-entry ROB limit progress at every width. Wider issue cannot bypass the oldest blocked instruction. The identical final states show that these structural stalls do not change the result.

4.7 Small ROB, reservation stations, and LSQ - issue widths 1 and 2

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 1

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F3 F1 F2	1	2-9	-	10	11
2	4	Add.d F4 F3 F1	2	11-13	-	14	15
3	8	Add R4 R2 R3	3	4-5	-	6	16
4	12	Addi R5 R4 1	5	7-8	-	9	17
5	16	Ld F5 0(R1)	12	13	14-16	17	18
6	20	Add.d F6 F5 F4	16	18-20	-	21	22
7	24	Sd F6 4(R1)	17	18	23-25	-	25
8	28	Ld F7 4(R1)	18	19	22*	23	26
9	32	Mult.d F8 F7 F2	19	24-31	-	32	33
10	36	Addi R6 R5 1	23	24-25	-	26	34
11	40	Add.d F9 F8 F1	26	33-35	-	36	37
12	44	Sd F9 8(R1)	27	28	38-40	-	40
13	48	Ld F10 8(R1)	34	35	37*	38	41
14	52	Sub R7 R6 R2	35	36-37	-	39	42

Table 4.7-1. 14 committed instructions; 42 total cycles. Source: results/resource_pressure_w1.json.

Issue width 2

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F3 F1 F2	1	2-9	-	10	11
2	4	Add.d F4 F3 F1	1	11-13	-	14	15
3	8	Add R4 R2 R3	2	3-4	-	5	16
4	12	Addi R5 R4 1	4	6-7	-	8	17
5	16	Ld F5 0(R1)	12	13	14-16	17	18
6	20	Add.d F6 F5 F4	16	18-20	-	21	22
7	24	Sd F6 4(R1)	17	18	23-25	-	25
8	28	Ld F7 4(R1)	18	19	22*	23	26
9	32	Mult.d F8 F7 F2	19	24-31	-	32	33
10	36	Addi R6 R5 1	23	24-25	-	26	34
11	40	Add.d F9 F8 F1	26	33-35	-	36	37
12	44	Sd F9 8(R1)	27	28	38-40	-	40
13	48	Ld F10 8(R1)	34	35	37*	38	41
14	52	Sub R7 R6 R2	35	36-37	-	39	42

Table 4.7-2. 14 committed instructions; 42 total cycles. Source: results/resource_pressure_w2.json.

4.7 Small ROB, reservation stations, and LSQ - issue widths 3 and 4

I show every committed dynamic instruction in program order. ID is the dynamic issue tag, and PC is the byte address in the input program. Missing IDs belong to squashed attempts. Stage intervals are inclusive; a dash means the instruction does not use that stage. An asterisk on MEM marks one-cycle store-to-load forwarding. Stores update memory at the end of their commit service, so their COMMIT equals MEM end; branches and stores do not broadcast on the CDB.

Issue width 3

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F3 F1 F2	1	2-9	-	10	11
2	4	Add.d F4 F3 F1	1	11-13	-	14	15
3	8	Add R4 R2 R3	1	2-3	-	4	16
4	12	Addi R5 R4 1	3	5-6	-	7	17
5	16	Ld F5 0(R1)	12	13	14-16	17	18
6	20	Add.d F6 F5 F4	16	18-20	-	21	22
7	24	Sd F6 4(R1)	17	18	23-25	-	25
8	28	Ld F7 4(R1)	18	19	22*	23	26
9	32	Mult.d F8 F7 F2	19	24-31	-	32	33
10	36	Addi R6 R5 1	23	24-25	-	26	34
11	40	Add.d F9 F8 F1	26	33-35	-	36	37
12	44	Sd F9 8(R1)	27	28	38-40	-	40
13	48	Ld F10 8(R1)	34	35	37*	38	41
14	52	Sub R7 R6 R2	35	36-37	-	39	42

Table 4.7-3. 14 committed instructions; 42 total cycles. Source: results/resource_pressure_w3.json.

Issue width 4

ID	PC	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	0	Mult.d F3 F1 F2	1	2-9	-	10	11
2	4	Add.d F4 F3 F1	1	11-13	-	14	15
3	8	Add R4 R2 R3	1	2-3	-	4	16
4	12	Addi R5 R4 1	3	5-6	-	7	17
5	16	Ld F5 0(R1)	12	13	14-16	17	18
6	20	Add.d F6 F5 F4	16	18-20	-	21	22
7	24	Sd F6 4(R1)	17	18	23-25	-	25
8	28	Ld F7 4(R1)	18	19	22*	23	26
9	32	Mult.d F8 F7 F2	19	24-31	-	32	33
10	36	Addi R6 R5 1	23	24-25	-	26	34
11	40	Add.d F9 F8 F1	26	33-35	-	36	37
12	44	Sd F9 8(R1)	27	28	38-40	-	40
13	48	Ld F10 8(R1)	34	35	37*	38	41
14	52	Sub R7 R6 R2	35	36-37	-	39	42

Table 4.7-4. 14 committed instructions; 42 total cycles. Source: results/resource_pressure_w4.json.

5 Verification, assumptions, and reproduction

I checked timing and architectural state separately. tests/oracle.py executes the ISA sequentially with its own parser and memory conversion; it does not calculate expected Tomasulo cycles. The deterministic tests check issue limits, latency, operand readiness, queue occupancy, and recovery timing directly.

Check	Result / evidence
Automated suite	55 test methods passed. results/test_log.txt records the complete run.
Seeded differential tests	120 configurations, comparing all 64 registers and all 64 memory words.
Benchmarks	7 programs x 4 issue widths = 28 passing configurations. Full text/JSON output is retained in results/.
Command line	Input parsing, width overrides, output files, missing files, invalid input, and cycle-limit handling are checked.
Additional stress review	1,600 configurations passed; 400 seeded 40-instruction programs at four widths. 63 memory replays exercised.
Precision regression	Independent constants and alternate conversion in the oracle check memory initialization, stores, forwarding, and later loads.
Review additions	RS head-of-line blocking; ROB-full stalls; early load-queue release; intervening-store replay exception; delayed-address memory programs.

Assumptions I made explicit

I interpret the 256-byte, 64-word memory as aligned four-byte words at byte addresses 0, 4, ..., 252. The assignment uses byte addresses, but its illustrative R1=10 and Ld F4,8(R1) would produce address 18. It does not define how an unaligned word access should work. My simulator rejects that address; supporting unaligned accesses would require a separate byte-level memory interpretation.

I use one in-order commit slot, release an arithmetic reservation station at EX start, and allow that slot to be reused by issue in the following cycle. A result broadcast in cycle n becomes an execution operand in cycle n+1. These timing choices are documented and tested, but the handout does not specify every same-cycle boundary. Forwarding shares the single memory-stage service.

I found no remaining failing cases in the included tests. I do not model caches, virtual memory, interrupts, architectural FP flags, or fixed-width integer overflow. These are not specified by the supplied assignment. The alignment ambiguity remains an assumption rather than an unqualified claim of compatibility with every possible interpretation.

5 Verification - recovery evidence and running the submission

I checked that recovery affects the visible program only when the responsible instruction survives. The following events are extracted from the width-4 branch and replay traces.

Branch recovery at width 4

Event	Cycle n	Discarded	First issue after n
branch misprediction	7	7	9
branch misprediction	27	12	29
branch misprediction	33	5	35

Memory replay at width 4

Event	Cycle n	Discarded	First issue after n
memory replay	20	5	22

For a branch misprediction, the first fetch is allowed at n+2. A resource stall can postpone actual issue further. Replay uses the same recovery delay in my implementation. I also have direct tests for the exact branch restart cycle and for preserving older uncommitted register mappings.

Reproducing my results

```
python3 -m unittest discover -s tests -v
python3 run_experiments.py
python3 -m tests.stress_review
python3 main.py benchmarks/corrected_starter.in --issue-width 4
python3 main.py benchmarks/branch_recovery.in --issue-width 4 \
  --show-squashed --output branch.txt --json branch.json
```

I used Python 3.12.14 on Linux. The simulator and tests need only the standard library. Run the commands from the repository root. The rerun time, command exit codes, and source-file hashes are recorded in results/validation.json.

The supplied starter loop contains Bne R1 R0 -12. Under word-offset branch semantics that points outside the program. I preserve the original input and use -8 in corrected_starter.in to repeat its full loop. I document this input correction instead of silently changing the meaning of branch offsets.

My conclusion

The wider issue stage works for the tested programs while preserving their architectural results. Its benefit is workload dependent: the best benchmark reduction was 10%, and several cases were limited by dependencies, memory, or recovery. The review strengthened both the implementation and the evidence: it fixed single-precision memory transfers and made every reported benchmark reproducible from its input and trace.